

Runtime

Version: 1.0

Purpose

This document defines the runtime architecture of Trading Platform Pro and its primary application, the Trading Cockpit.

The runtime is responsible for starting, operating, supervising and shutting down the application in a controlled and deterministic manner.

Runtime behaviour is operationally critical because the Trading Cockpit coordinates:

- the desktop user interface
- asynchronous application services
- broker connectivity
- market data connectivity
- market data subscriptions
- background workers
- persistence
- monitoring and health state

The runtime shall preserve explicit lifecycle ownership and controlled failure behaviour.

Objectives

The runtime shall provide:

- deterministic startup
- deterministic shutdown
- centralized lifecycle coordination
- explicit runtime state
- controlled service orchestration
- asynchronous task ownership
- broker lifecycle coordination
- market data lifecycle coordination
- UI responsiveness
- runtime monitoring
- health visibility
- fault isolation
- graceful degradation
- controlled failure handling

Runtime Principles

The runtime shall:

- start components in a defined order
- stop components in a defined order
- expose operational state
- own background task lifecycle
- avoid hidden unmanaged workers
- avoid blocking the UI thread
- avoid blocking the AsyncIO event loop
- support cancellation
- use explicit timeouts where appropriate
- preserve failure context
- prevent silent runtime degradation

Business rules do not belong in runtime infrastructure.

The runtime coordinates technical lifecycle.

Runtime State

Runtime state shall be explicit.
Initial runtime states may include:

- Bootstrapping
- Initializing
- Starting
- Operational
- Degraded
- Stopping
- Stopped
- Failed

State transitions shall remain controlled and observable.

Example:

```
```text
```

Bootstrapping

↓

Initializing

↓

Starting

↓

Operational

↓

Stopping

↓

Stopped

```
```
```

Failure may transition the runtime to:

```
```text
```

Degraded

```
```
```

or:

```
```text
```

Failed

```
```
```

depending on operational impact.

Runtime Lifecycle

The application lifecycle consists of the following phases:

```
```text
```

Bootstrap

↓

Configuration

↓

Logging

↓

Dependency Container

↓

Infrastructure Initialization

↓

Application Services

↓

Runtime Services

↓

Presentation Initialization

↓

Workspace Restoration

↓

Operational State

↓

Controlled Shutdown

...

Each phase shall have a defined responsibility.

Business workflows shall not execute before required initialization is complete.

---

## Bootstrap

Bootstrap is the first runtime phase.

Responsibilities include:

- identify startup context
- begin minimal startup diagnostics
- initiate configuration loading
- prepare application construction

Bootstrap shall not execute trading workflows.

Bootstrap failures shall prevent incomplete application startup.

---

## Configuration Phase

Configuration shall be loaded and validated before dependent services start.

The runtime shall determine:

- active profile
- PAPER or LIVE environment
- broker configuration
- market data configuration
- persistence configuration
- runtime configuration

Critical configuration failures shall fail startup.

LIVE environment context shall remain explicit.

---

## Logging Initialization

Structured logging shall be initialized before operational services start.

Startup logging shall record non-sensitive context such as:

- application version
- active profile
- PAPER or LIVE environment
- configured broker provider
- configured market data provider

Secrets shall never be logged.

After logging initialization, runtime failures shall be recorded through structured logging.

---

## Dependency Container

The dependency container shall be constructed after validated configuration is available.

The container may register:

- repositories
- broker adapters
- market data adapters
- application services

- runtime services
- monitoring services
- health services
- presentation services

Dependency registration shall remain deterministic.

Business components shall not retrieve arbitrary services from a global container.

---

## Infrastructure Initialization

Infrastructure shall be initialized before dependent application workflows.

Infrastructure initialization may include:

- persistence
- broker adapter
- market data adapter
- messaging
- runtime services
- monitoring
- health checks

Initialization does not automatically imply external connectivity.

Infrastructure initialization shall establish technical readiness for controlled startup.

---

## Persistence Initialization

Persistence initialization may include:

- database path validation
- database connection initialization
- schema validation
- repository readiness

Persistence failures affecting required business state shall prevent normal operational startup.

The application shall not silently switch to temporary persistence.

---

## Broker Runtime Lifecycle

Broker integration has an explicit runtime lifecycle.

Example:

```
```text
```

Initialized

↓

Connecting

↓

Connected

↓

Operational

```
```
```

Failure states may include:

- Disconnected
- Degraded
- Reconnecting
- Failed

Broker connectivity shall remain distinguishable from overall runtime state.

The application may remain operational with limited capability when broker connectivity is unavailable.

Operational impact shall be visible.

---

## Broker Startup

Broker startup may include:

1. Initialize broker adapter.
2. Validate broker environment context.
3. Start connection lifecycle.
4. Establish connection where configured.
5. Retrieve initial connection state.
6. Expose broker capability state.

A broker connection failure shall not be interpreted as successful broker readiness.

The runtime shall not infer LIVE readiness solely from a successful network connection.

---

## Broker Reconnection

Broker reconnection behaviour shall be explicit.

Reconnection policy shall consider:

- provider behaviour
- connection state
- current runtime state
- active operations
- duplicate execution risk

Broker reconnection shall not blindly repeat order submissions.

Connection recovery and order submission are separate concerns.

Reconnection attempts shall be observable.

---

## Market Data Runtime Lifecycle

Market data integration has an explicit runtime lifecycle.

Example:

```text

Initialized

↓

Connecting

↓

Connected

↓

Subscription Ready

```

Failure states may include:

- Disconnected
- Degraded
- Subscription Failed
- Failed

Market data state shall remain distinguishable from broker state.

---

## Market Data Startup

Market data startup may include:

1. Initialize market data adapter.
2. Establish provider connectivity where required.
3. Retrieve connection state.
4. Enable controlled subscription requests.
5. Expose market data capability state.

The runtime shall not create uncontrolled subscriptions during startup.

Subscriptions shall be created by defined application workflows.

---

## Market Data Subscription Lifecycle

Market data subscriptions require explicit ownership.

Subscription lifecycle:

```text

Requested

↓

Active

↓

Updating

↓

Closing

↓

Closed

```

Failure states may include:

- Failed
- Disconnected
- Stale

The owner of a subscription shall be identifiable.

Subscriptions shall be releasable.

Shutdown shall close active subscriptions before market data infrastructure is disposed.

---

## Application Service Initialization

Application services shall be initialized after required infrastructure dependencies are available.

Application services coordinate:

- use cases
- product workflows
- reconciliation workflows
- application state transitions

Initialization shall not automatically execute user trading actions.

Order submission requires an explicit application workflow.

---

## Runtime Services

Runtime services may include:

- Runtime Coordinator
- Task Supervisor
- Event Dispatcher
- Scheduler
- Health Monitoring
- Runtime Monitoring
- Clock Service
- Broker Connection Supervisor
- Market Data Connection Supervisor
- Background Workers

Only implemented services shall be registered.

Do not introduce runtime services for hypothetical future capabilities.

---

## Runtime Coordinator

The Runtime Coordinator owns the top-level technical lifecycle.

Responsibilities include:

- startup coordination
- runtime state transitions
- service startup order
- service shutdown order

- critical failure coordination

The Runtime Coordinator shall not contain trading business logic.

---

## Task Supervisor

The Task Supervisor owns long-running asynchronous background tasks.

Responsibilities include:

- task registration
- task identity
- task state
- cancellation
- failure observation
- shutdown coordination

Background tasks shall not be created without lifecycle ownership.

Avoid unmanaged:

```
```python
asyncio.create_task(...)
```
```

for long-running application services.

Created tasks shall be registered with an owning runtime component.

---

## AsyncIO Runtime

AsyncIO may be used for:

- broker communication
- market data communication
- subscriptions
- background workers
- scheduled technical work
- external integrations

AsyncIO operations shall:

- avoid blocking calls
- use explicit cancellation
- use timeouts where appropriate
- preserve exception context
- release resources

Blocking operations shall be isolated from the event loop.

---

## UI Runtime

The Trading Cockpit uses PySide6.

The UI thread shall remain responsive.

The UI thread shall not perform:

- broker network operations
- market data network operations
- blocking database operations
- long-running calculations
- blocking file operations

External and long-running work shall be coordinated through application and runtime services.

---

## UI and AsyncIO Coordination

PySide6 and AsyncIO coordination shall use an explicit integration mechanism.

The selected integration approach shall define:

- event loop ownership

- startup order
- shutdown order
- task scheduling
- exception propagation

Do not create independent competing event loops without explicit architecture justification.

UI callbacks shall not hide uncontrolled asynchronous task creation.

---

## Presentation Initialization

Presentation initialization occurs after required application and runtime services are available.

Presentation initialization may include:

- application shell
- main window
- workspace manager
- widget registry
- shared instrument context
- notification system
- command palette

The main window shall not directly initialize broker or market data adapters.

---

## Workspace Restoration

Workspace restoration occurs after presentation infrastructure is ready.

Workspace restoration may include:

- layout
- widget visibility
- widget position
- widget size
- widget-specific UI state

Workspace restoration shall not execute trading actions.

Invalid workspace state shall not prevent application startup.

If workspace restoration fails:

- record the failure
- expose operational information where relevant
- load a safe fallback workspace

---

## Operational State

The runtime enters `Operational` only after required startup phases complete.

Operational state does not imply that every external capability is available.

Example:

```
```text
```

```
Runtime: Operational
```

```
Broker: Disconnected
```

```
Market Data: Connected
```

```
```
```

The application may remain usable with degraded capability.

Capability state and runtime state shall remain separate.

---

## Degraded State

The runtime may enter `Degraded` when the application remains usable but one or more important capabilities are unavailable.

Examples:

- broker disconnected



- market data unavailable
- background service failed
- monitoring partially unavailable

Degraded state shall expose:

- affected capability
- operational impact
- recovery state

Degraded operation shall not silently enable unsafe fallback behaviour.

---

## Background Workers

Background workers require explicit lifecycle ownership.

Each worker shall define:

- purpose
- owner
- startup condition
- stop condition
- cancellation behaviour
- failure behaviour

Workers shall not silently restart unless restart policy is explicitly defined.

Worker failures shall be observable.

---

## Scheduler Runtime

The scheduler provides technical timing capabilities.

Scheduled runtime work may include:

- health checks
- maintenance
- technical refresh
- monitoring collection

The scheduler shall not contain trading strategy decisions.

Scheduled trading workflows require explicit Application ownership.

---

## Event Dispatcher Runtime

The Event Dispatcher may coordinate Domain, Application or operational events.

Event processing shall remain observable.

Critical event handler failures shall not be silently swallowed.

Business-critical workflows shall not depend on hidden event chains.

Shutdown shall stop new event intake before dispatcher disposal.

---

## Monitoring

Runtime monitoring may observe:

- runtime state
- service state
- task state
- broker connection state
- market data connection state
- subscription state
- scheduler state
- event processing state
- critical failures

Monitoring shall not modify trading behaviour.

Monitoring failures shall remain distinguishable from monitored capability failures.

---

## Health State

Health checks evaluate technical capability availability.

Health states may include:

- Healthy
- Degraded
- Unhealthy
- Unknown

Health checks may evaluate:

- persistence
- broker adapter
- market data adapter
- runtime workers
- event dispatcher

Health checks shall remain lightweight.

A health check shall not submit orders or mutate business state.

---

## Failure Classification

Runtime failures shall be classified by operational impact.

Example categories:

- Recoverable
- Degrading
- Critical

### Recoverable

The operation failed but runtime capability remains available.

Example:

- temporary technical refresh failure

### Degrading

A capability is unavailable but the application may continue.

Example:

- market data disconnected

### Critical

Safe continued operation is not possible.

Example:

- unrecoverable persistence failure affecting required business state

Critical failures shall trigger controlled shutdown where required.

---

## Failure Isolation

Runtime components shall isolate failures where practical.

A market data worker failure shall not automatically terminate the UI.

A presentation widget failure shall not automatically terminate broker connectivity.

Isolation shall not hide operational impact.

Failure boundaries shall remain explicit.

---

## Timeouts

Long-running technical operations shall use explicit timeout behaviour where appropriate.

Examples:

- broker connection
- market data connection
- external requests
- worker shutdown

Timeouts shall be configurable where justified.

A timeout shall remain distinguishable from:

- rejection
- cancellation
- disconnection
- validation failure

---

## Cancellation

Cancellation is part of normal runtime control.

Cancellation shall:

- be explicit
- propagate through owned operations where appropriate
- preserve valid state
- release resources
- remain observable for critical operations

Cancellation shall not be interpreted as successful external completion.

---

## Shutdown Request

Shutdown may be requested by:

- user action
- operating system application close
- controlled critical failure
- development or test control

Shutdown requests shall be coordinated by the Runtime Coordinator.

Multiple shutdown requests shall not create parallel shutdown sequences.

---

## Controlled Shutdown

Controlled shutdown shall follow a defined sequence.

Default shutdown sequence:

1. Enter `Stopping`.
2. Reject new user-initiated trading actions.
3. Stop new application workflow intake.
4. Stop new market data subscription requests.
5. Stop scheduler intake.
6. Stop new event intake where appropriate.
7. Cancel or complete owned background tasks according to policy.
8. Close active market data subscriptions.
9. Stop market data connection lifecycle.
10. Stop broker connection lifecycle.
11. Flush application state where required.
12. Persist workspace state.
13. Stop monitoring and health services.
14. Dispose runtime services.
15. Dispose infrastructure.
16. Flush structured logs.
17. Enter `Stopped`.

Shutdown shall remain observable.

---

## Trading Actions During Shutdown

After runtime enters `Stopping`:

- new order submission shall be rejected
- new order cancellation workflows shall follow explicit shutdown policy
- new trading decisions shall not trigger execution
- new market data subscriptions shall be rejected

The system shall not silently accept trading actions that cannot complete safely.

---

## Active Order Operations During Shutdown

Active order operations require explicit handling.

The runtime shall distinguish between:

- local workflow activity
- transmitted broker requests
- broker-acknowledged orders
- active broker orders

Shutdown shall not blindly cancel broker orders.

Broker order cancellation is a business-critical external action and requires explicit application policy.

The runtime shall preserve sufficient state for later reconciliation.

---

## Shutdown Timeouts

Shutdown operations shall use defined timeout behaviour where required.

Examples:

- background task cancellation
- subscription closure
- broker disconnect
- log flush

A shutdown timeout shall be logged.

Critical state required for reconciliation shall be preserved where technically possible.

The process shall not wait indefinitely for external systems.

---

## Unexpected Exceptions

Unexpected runtime exceptions shall:

- be recorded through structured logging
- preserve exception context
- identify the affected runtime component
- trigger failure classification

Unexpected exceptions shall not terminate the application silently.

Presentation of technical failure to the user shall communicate operational impact without exposing internal stack traces.

---

## Runtime Observability

Runtime state transitions shall be observable.

Relevant operational events may include:

- RuntimeBootstrapping
- RuntimeInitializing
- RuntimeStarting
- RuntimeOperational
- RuntimeDegraded

- RuntimeStopping
- RuntimeStopped
- RuntimeFailed

Technical event naming may differ from Domain Events.

Runtime events shall not be represented as Domain business events.

---

## PAPER Runtime

PAPER runtime shall preserve explicit PAPER execution context.

The UI and runtime diagnostics shall expose PAPER state.

PAPER mode may support:

- broker paper account
- simulated adapters in testing
- execution workflow validation

PAPER mode shall not silently use LIVE configuration.

---

## LIVE Runtime

LIVE runtime is operationally critical.

Before LIVE operational state, the runtime shall validate:

- LIVE profile
- LIVE trading environment
- required broker configuration
- required persistence state
- critical runtime dependencies

LIVE state shall remain visible in the Trading Cockpit.

Runtime degradation shall not silently switch execution environment.

---

## Runtime Testing

Runtime behaviour requires automated tests.

Tests shall verify where relevant:

- deterministic startup order
- startup failure handling
- runtime state transitions
- degraded state
- task ownership
- task failure handling
- cancellation
- broker connection failure
- market data connection failure
- subscription cleanup
- workspace restoration failure
- controlled shutdown order
- shutdown timeout
- repeated shutdown request
- PAPER and LIVE separation

Tests shall not require LIVE trading.

---

## Runtime Evolution

Runtime capabilities evolve with concrete application requirements.

Before introducing a runtime service:

1. Identify the technical requirement.
2. Identify lifecycle ownership.

3. Define startup behaviour.
  4. Define shutdown behaviour.
  5. Define cancellation behaviour.
  6. Define failure behaviour.
  7. Define monitoring requirements.
  8. Add automated tests.
  9. Update documentation.
- Avoid runtime services created only for hypothetical future scalability.
- 

## Runtime Review Checklist

Before introducing or changing runtime behaviour verify:

- technical requirement identified
  - runtime owner identified
  - startup order defined
  - shutdown order defined
  - runtime state impact evaluated
  - broker lifecycle impact evaluated
  - market data lifecycle impact evaluated
  - UI thread impact evaluated
  - AsyncIO event loop impact evaluated
  - background task ownership defined
  - cancellation defined
  - timeout behaviour defined
  - failure classification defined
  - degraded behaviour defined
  - active order impact evaluated
  - PAPER impact evaluated
  - LIVE impact evaluated
  - observability defined
  - automated tests added
  - documentation synchronized
- 

## Related Documents

- Product\_Vision.md
- Product\_Roadmap.md
- Project\_Overview.md
- Architecture.md
- Infrastructure.md
- Technical\_Specifications.md
- Configuration.md
- Logging.md
- Monitoring.md
- API\_Guidelines.md
- Testing\_Strategy.md
- AGENTS.md